

Engenharia de IA, SDD e BioByte Sentinela

Planejamento da nova apresentação — v1

Data: 14/09/2026
Duração: 120 minutos, incluindo intervalo
Público: desenvolvedores e arquitetos de software
Caso condutor: BioByte Sentinela — vigilância de IRAS, com foco em ICSAC
Arquivo da apresentação: `deck/output/apresentacao_biobyte_sdd_v1.pptx`

1. Decisão narrativa

Esta versão mantém as tecnologias da apresentação anterior, mas passa a utilizá-las sobre o mesmo problema técnico: o BioByte Sentinela.

O público primeiro recebe os fundamentos de IA e desenvolvimento agêntico. Em seguida conhece o domínio e a especificação do BioByte. Depois vê pequenos trechos de LangChain, LangGraph, CrewAI, AutoGen, MCP, Claude Code e Codex aplicados ao mesmo caso de uso. Por fim, acompanha a implementação real do pipeline LangNet:

BioByte/IRAS

- especificação
- agentes e tarefas
- UI Spec e protótipo
- Rede de Petri
- código e integrações
- testes e gates
- aplicação executável

O fio técnico principal é o caso `UC-004 – Classificar caso segundo NHSN`, conectado à consulta de microbiologia, à detecção de MDR, ao risco de Cox e à aprovação da CCIH.

2. Distinção essencial

Os exemplos de frameworks têm duas naturezas:

Tipo	Uso na apresentação
Exemplo didático	Pequeno trecho mostrando como o mesmo caso poderia ser modelado em LangChain, LangGraph, CrewAI ou AutoGen.
Artefato real	Documento, YAML, tarefa, protótipo, código, teste ou execução efetivamente produzidos pelo LangNet/BioByte.

Cada slide declara essa distinção. Não se deve afirmar que o BioByte foi executado em todos os frameworks apenas porque há exemplos comparativos.

3. Estrutura e tempo

Bloco	Tema	Tempo
0	Abertura e tese	5 min
1	Fundamentos de LLMs	12 min
2	Contexto, RAG, ferramentas e avaliação	11 min
3	O problema BioByte e sua especificação	12 min
4	Agentes e frameworks aplicados ao BioByte	18 min
—	Intervalo	5 min
5	SDD como método de engenharia	18 min
6	LangNet, agentes, tarefas e Redes de Petri	17 min
7	Demonstração real e fechamento, incluindo vídeo de 12 min	22 min
Total		120 min

4. Roteiro por bloco

Bloco 0 — Abertura e tese

Apresentar a tese:

O gargalo deixou de ser escrever código. Passou a ser especificar, orquestrar e verificar.

Explicar que a palestra responderá como uma equipe pode usar modelos, agentes e ambientes de desenvolvimento sem perder rastreabilidade.

Bloco 1 — Fundamentos de LLMs

Cobrir, sem excesso histórico:

- Transformer, tokens, atenção e contexto;
- decoder-only e previsão do próximo token;
- temperatura, custo, KV cache e prefix caching;
- modelos locais e APIs;
- limites de contexto e alucinação.

Exemplo curto: gerar uma classificação estruturada com baixa temperatura e schema explícito.

Bloco 2 — Contexto, RAG, ferramentas e avaliação

Usar o BioByte como primeiro exemplo técnico:

- recuperar critérios NHSN/ICSAC;
- consultar microbiologia por ferramenta;
- devolver evidência textual, classificação e confiança;
- diferenciar ausência de evidência, erro de dados e falha de serviço;

- validar a saída antes de persistir.

Mostrar MCP como contrato de ferramenta, sem apresentá-lo como mecanismo de governança completo.

Bloco 3 — O problema BioByte e a especificação

Apresentar o BioByte Sentinela como sistema de vigilância de IRAS, especialmente ICSAC:

```
fatores de risco → microbiologia → NHSN/ICSAC → MDR → alerta  
→ bundle/condução → risco de Cox → relatório e dashboard
```

Mostrar um trecho do documento de requisitos/especificação real, incluindo atores, requisitos, casos de uso e critérios de aceitação.

Caso central:

```
UC-004 – Classificar caso segundo NHSN  
Entrada: paciente, evolução, hemocultura, cateter e datas.  
Saída: confirmada, descartada ou pendente, com evidências e justificativa.
```

Bloco 4 — Agentes e frameworks no mesmo caso

Apresentar primeiro a definição operacional de agente e o laço básico. Depois usar o UC-004 em cada tecnologia:

- LangChain: composição de prompt, modelo, retriever e saída estruturada;
- LangGraph: estado, nós, transições, checkpoint e aprovação humana;
- CrewAI: Agent, Task, Crew, role, goal, backstory e expected_output;
- AutoGen/AG2: conversa entre classificador, revisor e humano;
- MCP: consultar_microbiologia com schema de entrada e saída;
- Claude Code e Codex: ambientes que leem o repositório, alteram arquivos, executam testes e revisam diffs.

Fechar com uma matriz comparativa: laço, estado, durabilidade, aprovação humana, observabilidade e acoplamento.

Bloco 5 — SDD como método

Mostrar a inversão:

```
requisito informal → código → documentação atrasada  
  
spec primária → plano → tarefas → código → testes → gate
```

Usar o UC-004 para demonstrar:

- requisito EARS;
- critério de aceitação;
- contrato de agente e tarefa;
- teste de borda da janela temporal;

- rastreabilidade até a implementação.

Apresentar UI Spec e protótipo como derivados da especificação, incluindo estados de loading, vazio, erro, falha externa e recuperação.

Bloco 6 — LangNet e Redes de Petri

Explicar as camadas reais:

```
LangNet
├─ requisitos e especificação
├─ modelo de dados
├─ UI Spec e protótipo
├─ agents.yaml e tasks.yaml
├─ Rede de Petri
├─ código/adapters
└─ testes e portões
```

Mostrar a semântica de execução:

- lugares como estados ou pontos de sincronização;
- transições como tarefas;
- tokens como estado de execução;
- lógica JavaScript no lugar;
- chamada ao agente/serviço Python via WebSocket;
- portão determinístico antes da publicação.

Bloco 7 — Demonstração e fechamento

O vídeo deve ter aproximadamente **12 minutos** e mostrar o pipeline real do BioByte, com a especificação como fonte e o resultado executável ao final. Esses 12 minutos estão incluídos nos 120 minutos totais da palestra. O vídeo precisa distinguir claramente:

1. documento de origem;
2. agentes e tarefas;
3. Rede de Petri;
4. protótipo/interface;
5. código e integrações;
6. teste ou falha real;
7. gate e rastreabilidade.

Distribuição sugerida do bloco final:

Parte	Tempo
Introdução da demonstração	1,5 min
Vídeo do pipeline BioByte	12 min
Comentário técnico após o vídeo	1,5 min
Conclusões	1,5 min
Referências e perguntas	6 min

Parte	Tempo
Total do bloco	22 min

Encerrar com três conclusões:

1. modelo é capacidade; método é engenharia;
2. agente sem gate propaga erro;
3. especificação executável transforma geração em processo auditável.

5. Artefatos que devem aparecer

Reais do BioByte/LangNet

- `brief_icsac.md`;
- requisitos e especificação do BioByte;
- `agents.yaml`;
- `tasks.yaml`;
- tarefa `classify_case_nhsn`;
- ferramenta `consultar_microbiologia`;
- telas de microbiologia, classificação, MDR, Cox e dashboard;
- protótipo e teste;
- Rede de Petri;
- gate de rastreabilidade.

Didáticos, mas baseados no mesmo contrato

- trecho LangChain;
- trecho LangGraph;
- trecho CrewAI;
- trecho AutoGen/AG2;
- contrato MCP.

6. Alinhamento com o ambiente atual

O slide de arquitetura operacional deve mencionar explicitamente, se estiverem ativos na execução da palestra:

- DeepSeek como modelo em uso;
- controle de tokens e custo;
- prefix caching quando aplicável;
- Cloud Code API e seu endpoint configurado;
- Claude Code/Codex como ambientes de desenvolvimento;
- Playwright para operar a interface;
- backend Python e WebSocket;
- JavaScript nos lugares da Rede de Petri;
- agentes, tarefas e protótipo do LangNet.

Valores de modelo, porta, endpoint e métricas devem ser confirmados no ensaio final. O planejamento não autoriza inventar uma integração que não esteja operacional.

7. Critérios de qualidade

- A plateia entende o BioByte antes dos exemplos de framework.
- Cada framework aparece com um trecho técnico curto e comparável.
- A apresentação mantém todas as tecnologias principais sem dar a todas o mesmo tempo.
- O público sabe quais trechos são didáticos e quais são execução real.
- A especificação aparece antes do código gerado.
- O vídeo fecha o arco especificação → execução → verificação.
- A numeração do planejamento, PowerPoint, notas e roteiro é idêntica.
- O vídeo é testado fora do PowerPoint e possui plano de contingência.

8. Itens para validar antes da palestra

- confirmar o modelo e endpoint realmente usados;
- confirmar se o Cloud Code API estará acessível;
- capturar telas reais do BioByte;
- substituir placeholders por artefatos reais;
- revisar números de benchmarks e licenças;
- conferir que a demo não afirma execução de um framework que não foi usado;
- ensaiar com cronômetro;
- preparar os slides de backup para detalhes de modelos, protocolos e frameworks.